

Git Commit Message Guidelines

For repositories with BugTracker commit validation enabled

CONFIDENTIAL: This document and the commit validation rules described herein are proprietary to Turneratech Pvt Ltd and intended solely for internal development teams and authorized contractors. Sharing outside Turneratech requires written approval.

1. Required commit format

To pass the repository commit validation workflow, every non-merge commit must satisfy **both** rules:

- **Rule A (first line):** The first line must start with a valid Bug ID followed by a colon, then a short description.
- **Rule B (author line anywhere):** The commit message must contain an author line on any line (can be on the first line or later).

Bug ID pattern

Bug IDs must use uppercase project keys and digits in this structure:

<PROJECT>-<NUMBER> where PROJECT is 2 to 10 letters (A-Z) and NUMBER is 1 to 6 digits.

Examples: WD-2, WD-0002, GRN-0005, SM-42, BH-0015

Author line pattern

Include an author line anywhere in the commit message using:

- **Author: <name>** (case-insensitive; whitespace tolerant).

Examples (valid)

Multi-line example (author later in the message):

WD-0003: fix legend size and emission labeling

- Updated legend sizing to avoid truncation
- Converted estimated values to calculated values where applicable
- Author: abhishek.m

Single-line example (author on the first line):

GRN-0005: improve light theme palette - Author: abhishek.m

2. Best practices for commit messages

- Keep the first line short and descriptive. Aim for 50-72 characters where practical.
- Use the imperative mood for the summary (e.g., "Fix", "Add", "Update", "Refactor").
- Explain "what" and "why" in the body. Avoid copying large logs; link to issues/PRs instead.
- Wrap body lines at ~72 characters for readability in terminals.
- Make commits atomic: one logical change per commit whenever possible.

- Avoid "WIP" or "temp" commits in shared branches. Use local commits and squash before pushing if needed.
- Reference related tickets in the body if helpful (in addition to the required Bug ID in the first line).
- Do not include secrets, credentials, private keys, customer data, or internal URLs that should not be shared.

3. How to fix a rejected commit

If validation fails, update your commit message locally and push again. Common commands:

Amend the most recent commit message:

```
git commit --amend
```

Reword older commits before pushing (interactive rebase):

```
git rebase -i <base-commit>  
# mark commits as 'reword'
```

After rewriting history on a shared branch, push with care:

```
git push --force-with-lease
```

Notes for pull requests

If your repository uses squash merges, ensure the final squash commit message also follows the required format. Merge commits created by GitHub may be skipped by the validator, but individual commits in PR branches are still validated.

Document owner: Turneratech Pvt Ltd

Classification: Confidential - Internal Use Only